

과제 보고서

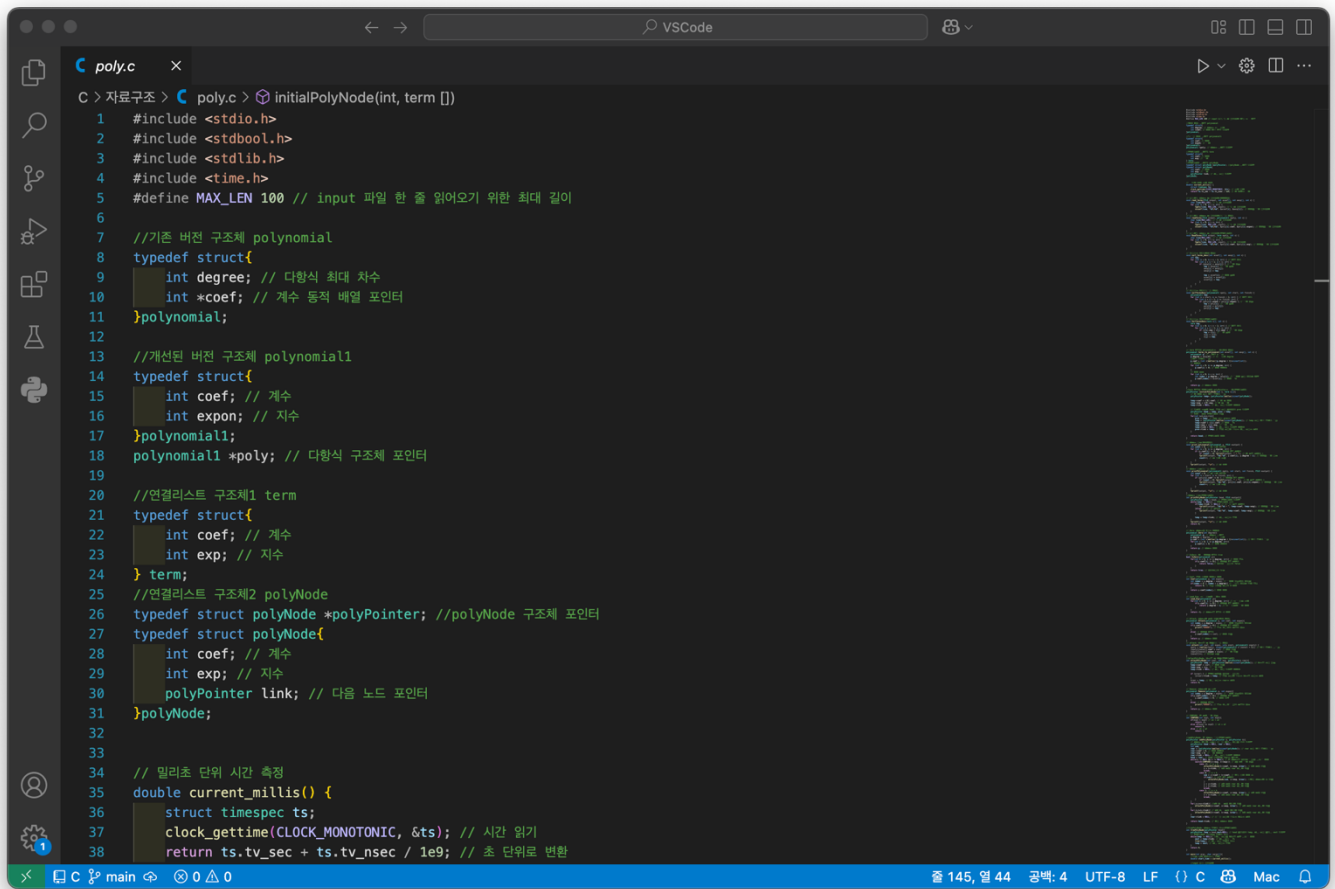
교과목명	자료구조
과제제목	Programming Assignment #1



제출월일	2025년 4월 15일
담당교수	이의종 교수님
제출자	성명: 김여민 학과: 소프트웨어학부 학년: 2학년 학번: 2024042053

01. 코드에 대한 설명(주석 참조)

- 1) macOS version



```
C > 자료구조 > C poly.c > initialPolyNode(int, term [])
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <stdlib.h>
4  #include <time.h>
5  #define MAX_LEN 100 // input 파일 한 줄 읽어오기 위한 최대 길이
6
7  //기존 버전 구조체 polynomial
8  typedef struct{
9      int degree; // 다항식 최대 차수
10     int *coef; // 계수 동적 배열 포인터
11 }polynomial;
12
13 //개선된 버전 구조체 polynomial1
14 typedef struct{
15     int coef; // 계수
16     int expon; // 지수
17 }polynomial1;
18 polynomial1 *poly; // 다항식 구조체 포인터
19
20 //연결리스트 구조체1 term
21 typedef struct{
22     int coef; // 계수
23     int exp; // 지수
24 } term;
25
26 //연결리스트 구조체2 polyNode
27 typedef struct polyNode *polyPointer; //polyNode 구조체 포인터
28 typedef struct polyNode{
29     int coef; // 계수
30     int exp; // 지수
31     polyPointer link; // 다음 노드 포인터
32 }polyNode;
33
34 // 밀리초 단위 시간 측정
35 double current_millis() {
36     struct timespec ts;
37     clock_gettime(CLOCK_MONOTONIC, &ts); // 시간 읽기
38     return ts.tv_sec + ts.tv_nsec / 1e9; // 초 단위로 변환
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
35 double current_millis() {
36 }
37
38
39 // 파일에서 다항식 항 읽어오기(초기버전)
40 void read_terms(FILE *input, int acoef[], int aexp[], int n) {
41     char line[MAX_LEN]; // 한 줄 읽어오기
42     for (int i = 0; i < n; i++) {
43         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
44         sscanf(line, "%d\t%d", &acoef[i], &aexp[i]); // 계수와 지수 읽어오기
45     }
46 }
47
48 // 파일에서 다항식 항 읽어오기(개선된 버전)
49 void readTerms(FILE *input, polynomial1 *poly, int n) {
50     char line[MAX_LEN]; // 한 줄 읽어오기
51     for (int i = 0; i < n; i++) {
52         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
53         sscanf(line, "%d\t%d", &poly[i].coef, &poly[i].expon); // 계수와 지수 읽어오기
54     }
55 }
56
57 // 파일에서 다항식 항 읽어오기(연결리스트)
58 void ReadTerms(FILE *input, term *poly, int n) {
59     char line[MAX_LEN]; // 한 줄 읽어오기
60     for (int i = 0; i < n; i++) {
61         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
62         sscanf(line, "%d\t%d", &poly[i].coef, &poly[i].exp); // 계수와 지수 읽어오기
63     }
64 }
65
66 // 내림차순 정렬(기존 버전)
67 void sort_terms_desc(int acoef[], int aexp[], int n) {
68     int tmp;
69     for (int i = 0; i < n - 1; i++) { // 버블 정렬
70         for (int j = i + 1; j < n; j++) {
71             if (aexp[i] < aexp[j]) { // 지수 비교
72                 tmp = acoef[i]; // 지수 교환
73                 acoef[i] = acoef[j];
74                 acoef[j] = tmp;
75             }
76         }
77     }
78 }
79
80 // 내림차순 정렬(개선된 버전)
81 void sortTermsDesc(polynomial1 *poly, int start, int finish) {
82     polynomial1 tmp;
83     for (int i = start; i <= finish - 1; i++) { // 버블 정렬
84         for (int j = i + 1; j <= finish; j++) {
85             if (poly[i].expon < poly[j].expon) { // 지수 비교
86                 tmp = poly[i]; // 지수 교환
87                 poly[i] = poly[j];
88                 poly[j] = tmp;
89             }
90         }
91     }
92 }
93
94 // 내림차순 정렬(연결리스트)
95 void SortTermsDesc(term s[], int n) {
96     term tmp;
97     for (int i = 0; i < n - 1; i++) { // 버블 정렬
98         for (int j = i + 1; j < n; j++) {
99             if (s[i].exp < s[j].exp) { // 지수 비교
100                 tmp = s[i]; // 지수 교환
101                 s[i] = s[j];
102                 s[j] = tmp;
103             }
104         }
105     }
106 }
107
108
109
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
67 void sort_terms_desc(int acoef[], int aexp[], int n) {
68     for (int i = 0; i < n - 1; i++) { // 버블 정렬
69         for (int j = i + 1; j < n; j++) {
70             if (aexp[i] < aexp[j]) { // 지수 비교
71                 tmp = acoef[i]; // 지수 교환
72                 acoef[i] = acoef[j];
73                 acoef[j] = tmp;
74             }
75         }
76     }
77 }
78
79 // 내림차순 정렬(개선된 버전)
80 void sortTermsDesc(polynomial1 *poly, int start, int finish) {
81     polynomial1 tmp;
82     for (int i = start; i <= finish - 1; i++) { // 버블 정렬
83         for (int j = i + 1; j <= finish; j++) {
84             if (poly[i].expon < poly[j].expon) { // 지수 비교
85                 tmp = poly[i]; // 지수 교환
86                 poly[i] = poly[j];
87                 poly[j] = tmp;
88             }
89         }
90     }
91 }
92
93 // 내림차순 정렬(연결리스트)
94 void SortTermsDesc(term s[], int n) {
95     term tmp;
96     for (int i = 0; i < n - 1; i++) { // 버블 정렬
97         for (int j = i + 1; j < n; j++) {
98             if (s[i].exp < s[j].exp) { // 지수 비교
99                 tmp = s[i]; // 지수 교환
100                 s[i] = s[j];
101                 s[j] = tmp;
102             }
103         }
104     }
105 }
106
107
108
109
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
110 // term 배열을 polynomial로 변환(기본 버전)
111 polynomial terms_to_polynomial(int acoef[], int aexp[], int n) {
112     polynomial p; // 구조체 p 선언
113     p.degree = aexp[0]; // 최대 차수 degree
114     // 동적 메모리 할당
115     p.coef = (int *)malloc((p.degree + 1)*sizeof(int));
116     // 초기화
117     for (int i = 0; i <= p.degree; i++) {
118         p.coef[i] = 0; // 계수 초기화
119     }
120     // 계수 입력
121     for (int i = 0; i < n; i++) {
122         int index = p.degree - aexp[i]; // 지수에 따라 인덱스 계산
123         p.coef[index] = acoef[i]; // 계수 저장
124     }
125     return p; // 다항식 반환
126 }
127
128 // term 배열을 연결리스트 polyPointer로 변환(연결리스트)
129 polyPointer initialPolyNode(int n, term s[]){
130     // 첫 번째 노드 동적 메모리 할당
131     polyPointer temp = (polyPointer)malloc(sizeof(polyNode));
132
133     temp->coef = s[0].coef; // 첫 항 계수
134     temp->exp = s[0].exp; // 첫 항 지수
135     temp->link = NULL; // 다음 노드 포인터 초기화
136
137     // 리스트 시작점 head, 이전 노드 가르키는 prev 포인터
138     polyPointer head = temp, prev = temp;
139     // 나머지 항들 연결리스트 추가
140     for(int i=1; i<n; ++i){
141         prev = temp; // temp 노드 prev로 설정
142         temp = (polyPointer)malloc(sizeof(polyNode)); // temp 노드 동적 메모리 할당
143         temp->coef = s[i].coef; // 계수 저장
144         temp->exp = s[i].exp; // 지수 저장
145         temp->link = NULL; // 다음 노드 포인터 초기화
146         prev->link = temp; // 이전 노드의 링크를 현재 노드로 설정
147     }
148     return head;
149 }
150
151 // 다항식 출력(기본버전)
152 void print_polynomial(polynomial p, FILE *output) {
153     int count = 0; // 항 개수 카운트
154     for (int i = 0; i <= p.degree; i++) {
155         if (p.coef[i] != 0) { // 계수가 0이 아니면
156             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
157             fprintf(output, "%dx^%d", p.coef[i], p.degree - i); // 계수와 지수 출력
158             count++; // 항 개수 증가
159         }
160     }
161     fprintf(output, "\n"); // 줄 바꿈
162 }
163
164 // 다항식 출력(개선된 버전)
165 void printPolynomial(polynomial *poly, int start, int finish, FILE *output) {
166     int count = 0; // 항 개수 카운트
167     for (int i = start; i <= finish; i++) {
168         if (poly[i].coef != 0) { // 계수가 0이 아니면
169             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
170             fprintf(output, "%dx^%d", poly[i].coef, poly[i].expon); // 계수와 지수 출력
171             count++; // 항 개수 증가
172         }
173     }
174     fprintf(output, "\n"); // 줄 바꿈
175 }
176
177 // 다항식 출력(연결리스트)
178 int printPolyNode(polyPointer head, FILE *output){
179     polyPointer temp = head; // 연결리스트 포인터
180     while(temp != NULL){ // 연결리스트 순회
181         if(temp->link != NULL){ // 마지막 항이 아니면
182             fprintf(output, "%dx^%d + ", temp->coef, temp->exp); // 계수와 지수 출력
183         }else{ // 마지막 항이면
184             fprintf(output, "%dx^%d", temp->coef, temp->exp); // 계수와 지수 출력
185         }
186         temp = temp->link;
187     }
188     return 1;
189 }
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
129 polyPointer initialPolyNode(int n, term s[]){
130     // 첫 번째 노드 동적 메모리 할당
131     polyPointer temp = (polyPointer)malloc(sizeof(polyNode));
132
133     temp->coef = s[0].coef; // 첫 항 계수
134     temp->exp = s[0].exp; // 첫 항 지수
135     temp->link = NULL; // 다음 노드 포인터 초기화
136
137     // 리스트 시작점 head, 이전 노드 가르키는 prev 포인터
138     polyPointer head = temp, prev = temp;
139     // 나머지 항들 연결리스트 추가
140     for(int i=1; i<n; ++i){
141         prev = temp; // temp 노드 prev로 설정
142         temp = (polyPointer)malloc(sizeof(polyNode)); // temp 노드 동적 메모리 할당
143         temp->coef = s[i].coef; // 계수 저장
144         temp->exp = s[i].exp; // 지수 저장
145         temp->link = NULL; // 다음 노드 포인터 초기화
146         prev->link = temp; // 이전 노드의 링크를 현재 노드로 설정
147     }
148     return head;
149 }
150
151 // 다항식 출력(기본버전)
152 void print_polynomial(polynomial p, FILE *output) {
153     int count = 0; // 항 개수 카운트
154     for (int i = 0; i <= p.degree; i++) {
155         if (p.coef[i] != 0) { // 계수가 0이 아니면
156             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
157             fprintf(output, "%dx^%d", p.coef[i], p.degree - i); // 계수와 지수 출력
158             count++; // 항 개수 증가
159         }
160     }
161     fprintf(output, "\n"); // 줄 바꿈
162 }
163
164 // 다항식 출력(개선된 버전)
165 void printPolynomial(polynomial *poly, int start, int finish, FILE *output) {
166     int count = 0; // 항 개수 카운트
167     for (int i = start; i <= finish; i++) {
168         if (poly[i].coef != 0) { // 계수가 0이 아니면
169             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
170             fprintf(output, "%dx^%d", poly[i].coef, poly[i].expon); // 계수와 지수 출력
171             count++; // 항 개수 증가
172         }
173     }
174     fprintf(output, "\n"); // 줄 바꿈
175 }
176
177 // 다항식 출력(연결리스트)
178 int printPolyNode(polyPointer head, FILE *output){
179     polyPointer temp = head; // 연결리스트 포인터
180     while(temp != NULL){ // 연결리스트 순회
181         if(temp->link != NULL){ // 마지막 항이 아니면
182             fprintf(output, "%dx^%d + ", temp->coef, temp->exp); // 계수와 지수 출력
183         }else{ // 마지막 항이면
184             fprintf(output, "%dx^%d", temp->coef, temp->exp); // 계수와 지수 출력
185         }
186         temp = temp->link;
187     }
188     return 1;
189 }
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
177 int printPolyNode(polyPointer head, FILE *output){
178     while(temp != NULL){ // 연결리스트 순회
179         // 마지막 항이면
180         temp = temp->link; // 다음 노드로 이동
181     }
182     fprintf(output, "\n"); // 줄 바꿈
183     return 0;
184 }
185
186 // Zero: 다항식을 0으로 초기화
187 polynomial Zero(int degree){
188     polynomial p; // 다항식 구조체
189     p.degree = degree; // 최대 차수
190     p.coef = (int *)malloc((p.degree + 1)*sizeof(int)); // 동적 메모리 할당
191     for(int i = 0; i <= p.degree; i++){
192         p.coef[i] = 0; // 계수 초기화
193     }
194     return p; // 다항식 반환
195 }
196
197 // IsZero: 모든 계수가 0이면 true
198 bool IsZero(polynomial p){
199     for(int i = 0; i <= p.degree; i++){ // 계수 체크
200         if(p.coef[i] != 0){ // 계수가 0이 아니면
201             return false; // 비어있지 않으면 false
202         }
203     }
204     return true; // 비어있으면 true
205 }
206
207 // Coef: 특정 차수의 계수를 반환
208 int Coef(polynomial p, int expon){
209     int index = p.degree - expon; // 지수에 해당하는 인덱스
210     if(index < 0 || index > p.degree){ // 유효한 인덱스 범위 체크
211         return 0; // 해당 차수가 없으면 0 반환
212     }
213     return p.coef[index]; // 계수 반환
214 }
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
214 int Coef(polynomial p, int expon){
215 }
216
217 // Lead_Exp: 최고 차항의 지수를 반환
218 int Lead_Exp(polynomial p){
219     for(int i = 0; i <= p.degree; i++){ // 최고 차항 찾기
220         if(p.coef[i] != 0){ // 계수가 0이 아니면
221             return p.degree - i; // 최고 차항의 지수 반환
222         }
223     }
224     return -1; // 다항식이 0이면 -1 반환
225 }
226
227 // Attach: 다항식에 항을 추가(기존 버전)
228 polynomial Attach(polynomial p, int coef, int expon){
229     int index = p.degree - expon; // 지수에 해당하는 인덱스
230     if(p.coef[index] != 0){ // 계수가 0이 아니면
231         printf("ERROR"); // 이미 존재하는 항이면 오류
232     }
233     else{ // 계수가 0이면
234         p.coef[index] = coef; // 계수 추가
235     }
236     return p; // 다항식 반환
237 }
238
239 // attach: 새로운 항 추가(개선된 버전)
240 void attach(int coef, int expon, int* avail, polynomial **poly) {
241     *poly = realloc(*poly, sizeof(polynomial) * (*avail + 1)); // 동적 메모리 재할당
242     (*poly)[*avail].coef = coef; // 계수 추가
243     (*poly)[*avail].expon = expon; // 지수 추가
244     (*avail)++; // 인덱스 증가
245 }
246
247 // attachPolyNode: 새로운 항 추가(연결리스트)
248 int attachPolyNode(int coef, int exp, polyPointer* rear){
249     polyPointer temp = (polyPointer)malloc(sizeof(polyNode)); // 새로운 노드 생성
250     temp->coef = coef; // 계수 추가
251     temp->exp = exp; // 지수 추가
252     temp->link = NULL; // 다음 노드 포인터 초기화
253 }
```

```
poly.c
C > 자료구조 > C poly.c > initialPolyNode(int, term [])
251 int attachPolyNode(int coef, int exp, polyPointer* rear){
252     if (*rear) { // 연결리스트가 비어있지 않으면
253         (*rear)->link = temp; // 이전 노드의 링크를 새로운 노드로 설정
254     }
255     *rear = temp; // 현재 노드를 rear로 설정
256     return 0;
257 }
258
259 // Remove: 다항식에 항 제거
260 polynomial Remove(polynomial p, int expon){
261     int index = p.degree - expon; // 지수에 해당하는 인덱스
262     if(p.coef[index] != 0){ // 계수가 0이 아니면
263         p.coef[index] = 0; // 계수 제거
264     }
265     else{ // 계수가 0이면
266         printf("ERROR"); // 이미 존재하지 않는 항이면 오류
267     }
268     return p; // 다항식 반환
269 }
270
271 // COMPARE: 두 항의 지수 비교
272 int COMPARE(int exp1, int exp2){
273     if(exp1 < exp2) // a1 < a2
274         return -1;
275     else if(exp1 == exp2) // a1 = a2
276         return 0;
277     else // a1 > a2
278         return 1;
279 }
280
281 //addPolyNode: 두 다항식 덧셈(연결리스트)
282 polyPointer addPolyNode(polyPointer a, polyPointer b){
283     // 다항식 결과의 시작과 마지막 가리킬 헤드와 리어 포인터
284     polyPointer head = NULL, rear = NULL;
285     int sum;
286     rear = (polyPointer)malloc(sizeof(polyNode)); // rear 노드 동적 메모리 할당
287     rear->coef = 0; // 계수 초기화
288     rear->exp = 0; // 지수 초기화
289 }
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
```

```
poly.c
C > 자료구조 > poly.c > main(int, char *[])
328 int freePolyNode(polyPointer head){
331     while(temp != NULL){ //현재 노드가 NULL이 아닐 때까지 반복
332         next = temp->link; // 다음 노드 저장
333         free(temp); // 현재 노드 메모리 해제
334         temp = next; // 다음 노드로 이동
335     }
336     return 0;
337 }
338
339 int main(int argc, char *argv[]){
340     //시작 시간 측정(파일 열기)
341     double start_time = current_millis();
342
343     //input 파일 읽어오기
344     FILE *input = fopen(argv[1], "r");
345     char line[MAX_LEN];
346     //output 파일 열기
347     FILE *output = fopen(argv[2], "w");
348
349     //종료 시간 측정(파일 열기)
350     double end_time = current_millis();
351
352     //-----기존 버전 시작
353
354     //시작 시간 측정(기존 버전)
355     double start_time0 = current_millis();
356
357     //첫 줄 읽기: 지수 개수 2개
358     fgets(line, MAX_LEN, input);
359     int n1, n2;
360     sscanf(line, "%d\t%d", &n1, &n2);
361
362     //배열 a1coef, a1exp 선언
363     int a1coef[n1];
364     int a1exp[n1];
365     //배열 a2coef, a2exp 선언
366     int a2coef[n2];
367     int a2exp[n2];
368
369     //구조체 a1 초기화 후 s1 저장
370     polynomial a1 = terms_to_polynomial(a1coef, a1exp, n1);
371     //구조체 a2 초기화 후 s2 저장
372     polynomial a2 = terms_to_polynomial(a2coef, a2exp, n2);
373
374     //다항식 a1 출력
375     print_polynomial(a1, output);
376     //다항식 a2 출력
377     print_polynomial(a2, output);
378
379     //구조체 d 선언
380     polynomial d;
381     //다항식 덧셈(초기버전)
382     d = Zero(a1.degree > a2.degree ? a1.degree : a2.degree);
383     int sum = 0;
384     while(!IsZero(a1) && !IsZero(a2)){ //두 다항식이 비어있지 않을 때까지 반복
385         switch (COMPARE(Lead_Exp(a1), Lead_Exp(a2))){ // a1과 a2의 최고차항 지수 비교
386             case -1: // a1 < a2
387                 d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
388                 a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
389                 break;
390             case 0: // a1 = a2
391                 sum = Coef(a1, Lead_Exp(a1)) + Coef(a2, Lead_Exp(a2)); // 동일 차수 계수 합
392                 if(sum != 0){ // 합이 0이 아니면
393                     d = Attach(d, sum, Lead_Exp(a1));
394                     a1 = Remove(a1, Lead_Exp(a1));
395                     a2 = Remove(a2, Lead_Exp(a2));
396                 }
397             case 1: // a1 > a2
398                 d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
399                 a1 = Remove(a1, Lead_Exp(a1));
400                 break;
401         }
402     }
403     //다항식 d 출력
404     print_polynomial(d, output);
405
406     //종료 시간 측정
407     double end_time0 = current_millis();
408     printf("실행 시간: %f\n", end_time0 - start_time0);
409
410     //파일 닫기
411     fclose(input);
412     fclose(output);
413
414     return 0;
415 }
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
339 int main(int argc, char *argv[]){
368     int a2exp[n2];
369
370     // n1 줄들 읽어와 s1 저장
371     read_terms(input, a1coef, a1exp, n1);
372     // n2 줄들 읽어와 s2 저장
373     read_terms(input, a2coef, a2exp, n2);
374
375     //배열 s1 내림차순 정렬
376     sort_terms_desc(a1coef, a1exp, n1);
377     //배열 s2 내림차순 정렬
378     sort_terms_desc(a2coef, a2exp, n2);
379
380     //구조체 a1 초기화 후 s1 저장
381     polynomial a1 = terms_to_polynomial(a1coef, a1exp, n1);
382     //구조체 a2 초기화 후 s2 저장
383     polynomial a2 = terms_to_polynomial(a2coef, a2exp, n2);
384
385     //다항식 a1 출력
386     print_polynomial(a1, output);
387     //다항식 a2 출력
388     print_polynomial(a2, output);
389
390     //구조체 d 선언
391     polynomial d;
392     //다항식 덧셈(초기버전)
393     d = Zero(a1.degree > a2.degree ? a1.degree : a2.degree);
394     int sum = 0;
395     while(!IsZero(a1) && !IsZero(a2)){ //두 다항식이 비어있지 않을 때까지 반복
396         switch (COMPARE(Lead_Exp(a1), Lead_Exp(a2))){ // a1과 a2의 최고차항 지수 비교
397             case -1: // a1 < a2
398                 d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
399                 a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
400                 break;
401             case 0: // a1 = a2
402                 sum = Coef(a1, Lead_Exp(a1)) + Coef(a2, Lead_Exp(a2)); // 동일 차수 계수 합
403                 if(sum != 0){ // 합이 0이 아니면
404                     d = Attach(d, sum, Lead_Exp(a1));
405                     a1 = Remove(a1, Lead_Exp(a1));
406                     a2 = Remove(a2, Lead_Exp(a2));
407                 }
408             case 1: // a1 > a2
409                 d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
410                 a1 = Remove(a1, Lead_Exp(a1));
411                 break;
412         }
413     }
414     //다항식 d 출력
415     print_polynomial(d, output);
416
417     //종료 시간 측정
418     double end_time0 = current_millis();
419     printf("실행 시간: %f\n", end_time0 - start_time0);
420
421     //파일 닫기
422     fclose(input);
423     fclose(output);
424
425     return 0;
426 }
```

```
poly.c
C > 자료구조 > poly.c > initialPolyNode(int, term [])
339 int main(int argc, char *argv[]) {
340     while(!IsZero(a1) && !IsZero(a2)) { // 두 다항식이 비어있지 않을 때까지 반복
341         switch (COMPARE(Lead_Exp(a1), Lead_Exp(a2))) { // a1과 a2의 최고차항 지수 비교
342             if(sum != 0) { // 합이 0이 아니면
343                 d = Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
344             }
345             else { // 합이 0이면
346                 d = Attach(d, 0, Lead_Exp(a1)); // d에 0 추가
347             }
348             if(sum) { // 합이 0이 아니면
349                 Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
350                 a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
351                 a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
352             }
353             break;
354             case 1: // a1 > a2
355                 d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
356                 a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
357                 break;
358         }
359     }
360     // a1 또는 a2 나머지 항을 d에 삽입
361     while(!IsZero(a1)) { // a1이 비어있지 않으면
362         d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
363         a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
364     }
365     while(!IsZero(a2)) { // a2가 비어있지 않으면
366         d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
367         a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
368     }
369     // 다항식 d 출력
370     print_polynomial(d, output);
371 }
372 // 종료 시간 측정(기준 버전)
373 double end_time0 = current_millis();
374 //-----기준 버전 종료
```

```
poly.c
C > 자료구조 > poly.c > main(int, char * [])
339 int main(int argc, char *argv[]) {
340     //-----개선된 버전 시작
341     //시작 시간 측정(기준 버전)
342     double start_time1 = current_millis();
343     //파일 포인터 초기화
344     rewind(input);
345     //첫 줄 읽기: 지수 개수 2개
346     fgets(line, MAX_LEN, input);
347     sscanf(line, "%d\t%d", &n1, &n2);
348     //구조체 poly 선언
349     poly = (polynomial1 *) malloc(sizeof(polynomial1) * (n1 + n2 + 1));
350     // n1과 n2 중 둘 중 하나와 poly 저장
351     readTerms(input, poly, n1+n2);
352     //구조체 s1 내림차순 정렬
353     sortTermsDesc(poly, 0, n1-1);
354     //구조체 s2 내림차순 정렬
355     sortTermsDesc(poly, n1, n1+n2-1);
356     //다항식 a1 출력
357     printPolynomial(poly, 0, n1-1, output);
358     //다항식 a2 출력
359     printPolynomial(poly, n1, n1+n2-1, output);
360     //다항식 덧셈(개선된 버전)
361     // starta, finisha, startb, finishb 선언
362     int starta = 0; // 첫번째 다항식 시작 인덱스
363     int finisha = n1 - 1; // 첫번째 다항식 끝 인덱스
364     int startb = n1; // 두번째 다항식 시작 인덱스
365     int finishb = n1 + n2 - 1; // 두번째 다항식 끝 인덱스
366     int *avail = (int *) malloc(sizeof(int)); // 새로운 항 저장할 위치 포인터 동적 메모리 할당
```

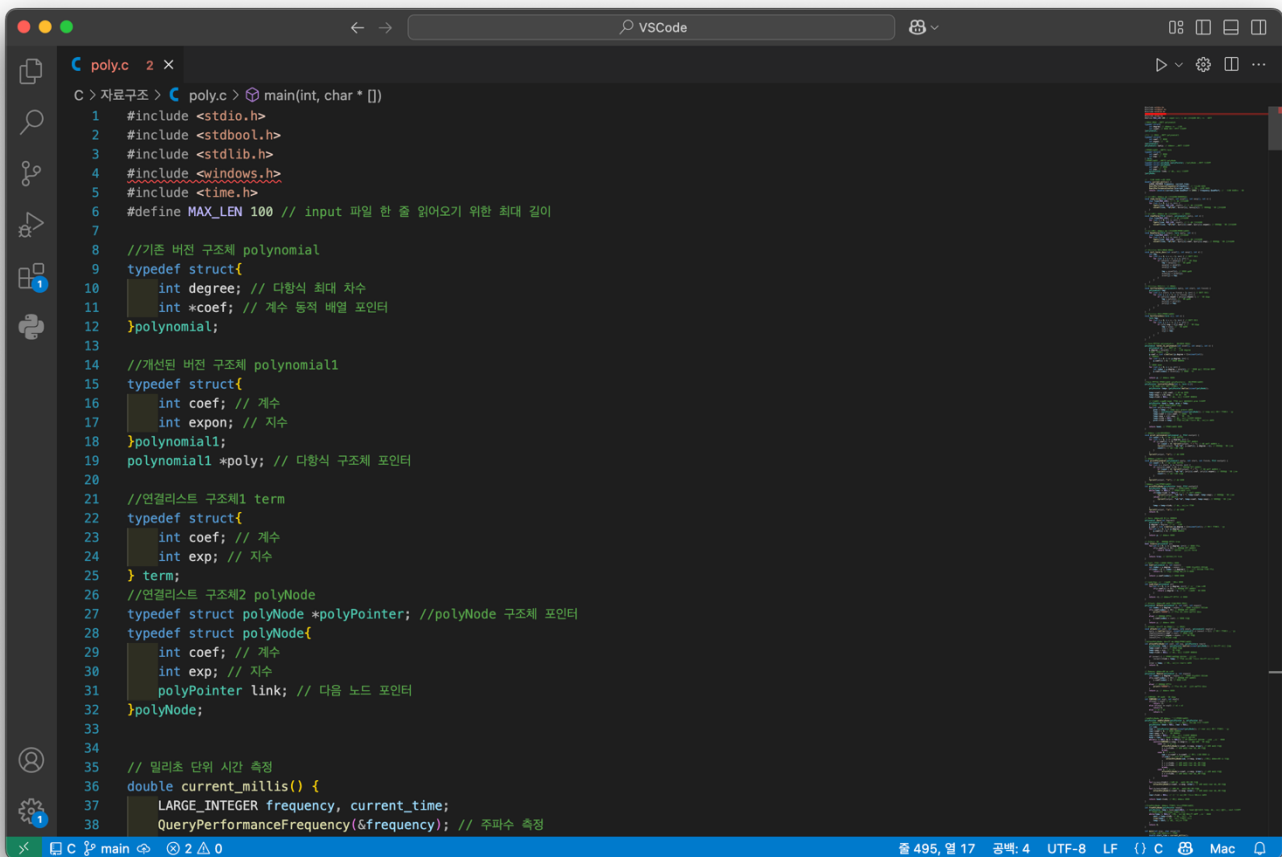
```
poly.c  output.txt
C > 자료구조 > C poly.c > main(int, char * [])
339 int main(int argc, char *argv[]){
476     *avail = n1 + n2; // 새로운 항 저장 위치 = 결과 다항식 시작 위치
477     int finished; // 결과 다항식 끝 위치
478
479     // 다항식 덧셈
480     float coefficient; // 계수
481     int startd = *avail; // 결과 다항식 시작 인덱스
482     while(starta <= finisha && startb <= finishb){ // 두 다항식이 끝날 때까지 반복
483         switch(COMPARE(poly[starta].expon, poly[startb].expon)){ // 지수 비교
484             case -1: // a.expon < b.expon
485                 attach(poly[startb].coef, poly[startb].expon, avail, &poly); // b의 항을 결과에 추가
486                 startb++; // b 다음 항으로 이동
487                 break;
488             case 0: // a.expon == b.expon
489                 coefficient = poly[starta].coef + poly[startb].coef; // 동일 지수 계수 합
490                 if(coefficient) // 합이 0이 아니면
491                     attach(coefficient, poly[starta].expon, avail, &poly); // 합한 계수를 결과에 추가
492                 starta++; // a 다음 항으로 이동
493                 startb++; // b 다음 항으로 이동
494                 break;
495             case 1: // a.expon > b.expon
496                 attach(poly[starta].coef, poly[starta].expon, avail, &poly); // a의 항을 결과에 추가
497                 starta++; // a 다음 항으로 이동
498                 break;
499         }
500     }
501     //a와 b 나머지 항 추가
502     for(; starta <= finisha; starta++){ // a에 남은 항을 결과에 추가
503         attach(poly[starta].coef, poly[starta].expon, avail, &poly);
504     }
505     for(; startb <= finishb; startb++){ // b에 남은 항을 결과에 추가
506         attach(poly[startb].coef, poly[startb].expon, avail, &poly);
507     }
508     finished = *avail - 1; // 결과 다항식 끝 인덱스
509     //다항식 d 출력
510     printPolynomial(poly, startd, finished, output);
511 }
512
```

```
poly.c  output.txt
C > 자료구조 > C poly.c > main(int, char * [])
339 int main(int argc, char *argv[]){
513
514     // 종료 시간 측정
515     double end_time1 = current_millis();
516
517     //-----개선된 버전 종료
518
519     //-----연결리스트 버전 시작
520
521     //시작 시간 측정(기준 버전)
522     double start_time2 = current_millis();
523
524     //파일 포인터 초기화
525     rewind(input);
526     //첫 줄 읽기: 지수 개수 2개
527     fgets(line, MAX_LEN, input);
528     sscanf(line, "%d\t%d", &n1, &n2);
529
530     //구조체 s1, s2 선언
531     term ls1[n1];
532     term ls2[n2];
533
534     // n1 줄들 읽어와 s1 저장
535     ReadTerms(input, ls1, n1);
536     // n2 줄들 읽어와 s2 저장
537     ReadTerms(input, ls2, n2);
538
539     //구조체 s1 내림차순 정렬
540     SortTermsDesc(ls1, n1);
541     //구조체 s2 내림차순 정렬
542     SortTermsDesc(ls2, n2);
543
544     //연결리스트 a, b 선언
545     polyPointer la, lb;
546
547     //구조체 a 초기화 후 s1 저장
548     la = initialPolyNode(n1, ls1);
549 }
```

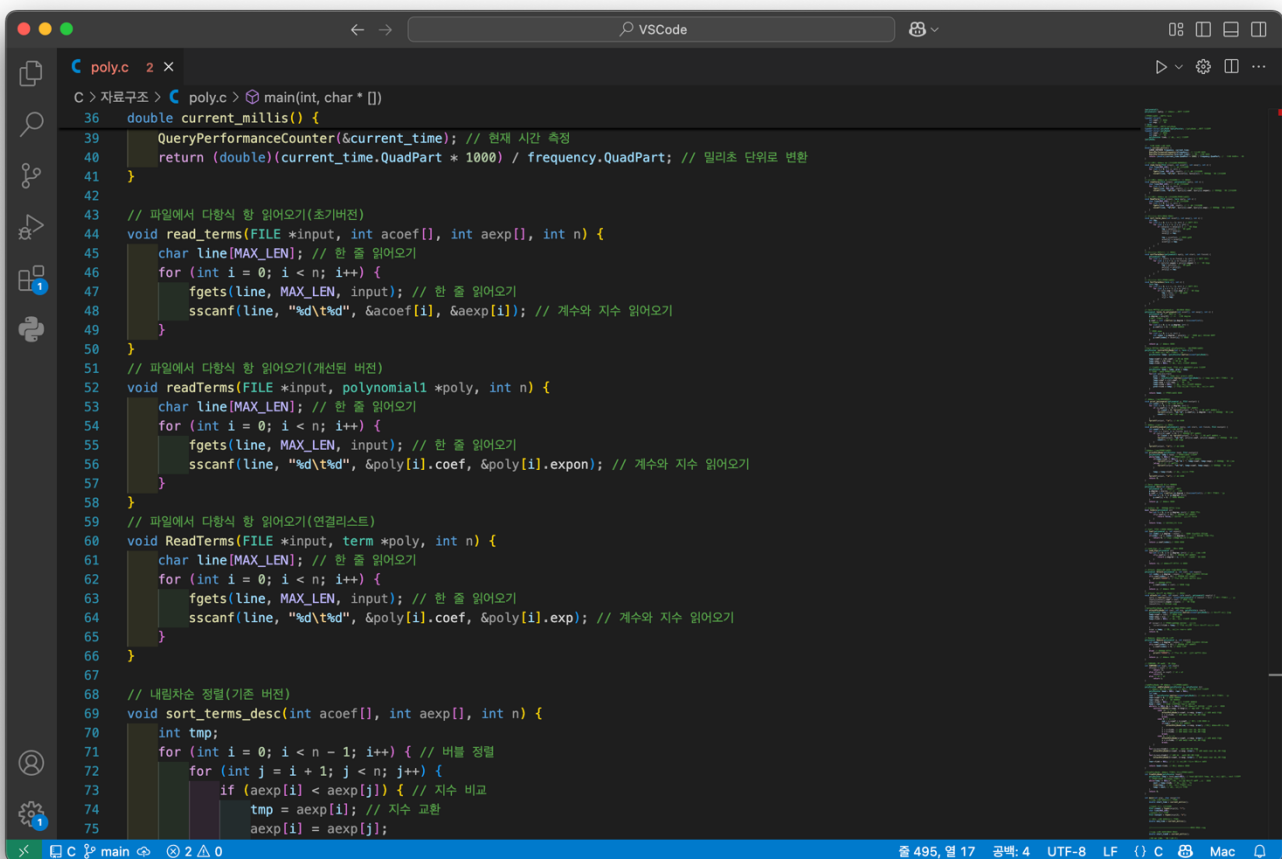
```
poly.c  x  output.txt
C > 자료구조 > C poly.c > main(int, char * [])
339 int main(int argc, char *argv[]){
550     //구조체 b 초기화 후 s2 저장
551     lb = initialPolyNode(n2, ls2);
552
553     //다항식 a 출력
554     printPolyNode(la, output);
555     //다항식 b 출력
556     printPolyNode(lb, output);
557     //다항식 덧셈 (연결리스트)
558     polyPointer c = addPolyNode(la, lb);
559     //다항식 c 출력
560     printPolyNode(c, output);
561
562     // 종료 시간 측정
563     double end_time2 = current_millis();
564
565     //-----연결리스트 종료
566
567     //수행시간 기록
568     // 수행 시간 기록(기준 버전)
569     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time0 - start_time0));
570     // 수행 시간 기록(개선된 버전)
571     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time1 - start_time1));
572     // 수행 시간 기록(연결리스트)
573     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time2 - start_time2));
574
575     //메모리 해제(malloc)
576     // 메모리 해제(기준 버전)
577     free(a1.coef);
578     free(a2.coef);
579     free(d.coef);
580     // 메모리 해제(개선된 버전)
581     free(avail);
582     free(poly);
583     //메모리 해제(연결리스트)
584     freePolyNode(la);
585     freePolyNode(lb);
586     freePolyNode(c);
```

```
poly.c  x  output.txt
C > 자료구조 > C poly.c > main(int, char * [])
339 int main(int argc, char *argv[]){
587
588     //input 파일 닫기
589     fclose(input);
590     //output 파일 닫기
591     fclose(output);
592     return 0;
593
594 }
```

- 2) window version



```
C > 자료구조 > C poly.c > main(int, char * [])
1 #include <stdio.h>
2 #include <stdbool.h>
3 #include <stdlib.h>
4 #include <windows.h>
5 #include <time.h>
6 #define MAX_LEN 100 // input 파일 한 줄 읽어오기 위한 최대 길이
7
8 //기존 버전 구조체 polynomial
9 typedef struct{
10     int degree; // 다항식 최대 차수
11     int *coef; // 계수 동적 배열 포인터
12 }polynomial;
13
14 //개선된 버전 구조체 polynomial1
15 typedef struct{
16     int coef; // 계수
17     int expon; // 지수
18 }polynomial1;
19 polynomial1 *poly; // 다항식 구조체 포인터
20
21 //연결리스트 구조체1 term
22 typedef struct{
23     int coef; // 계수
24     int exp; // 지수
25 } term;
26
27 //연결리스트 구조체2 polyNode
28 typedef struct polyNode *polyPointer; //polyNode 구조체 포인터
29 typedef struct polyNode{
30     int coef; // 계수
31     int exp; // 지수
32     polyPointer link; // 다음 노드 포인터
33 }polyNode;
34
35 // 밀리초 단위 시간 측정
36 double current_millis() {
37     LARGE_INTEGER frequency, current_time;
38     QueryPerformanceFrequency(&frequency); // 주파수 측정
```



```
36 double current_millis() {
37     QueryPerformanceCounter(&current_time); // 현재 시간 측정
38     return (double)(current_time.QuadPart * 1000) / frequency.QuadPart; // 밀리초 단위로 변환
39 }
40
41 // 파일에서 다항식 항 읽어오기(초기버전)
42 void read_terms(FILE *input, int acoef[], int aexp[], int n) {
43     char line[MAX_LEN]; // 한 줄 읽어오기
44     for (int i = 0; i < n; i++) {
45         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
46         sscanf(line, "%d\t%d", &acoef[i], &aexp[i]); // 계수와 지수 읽어오기
47     }
48 }
49
50 // 파일에서 다항식 항 읽어오기(개선된 버전)
51 void readTerms(FILE *input, polynomial1 *poly, int n) {
52     char line[MAX_LEN]; // 한 줄 읽어오기
53     for (int i = 0; i < n; i++) {
54         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
55         sscanf(line, "%d\t%d", &poly[i].coef, &poly[i].expon); // 계수와 지수 읽어오기
56     }
57 }
58
59 // 파일에서 다항식 항 읽어오기(연결리스트)
60 void ReadTerms(FILE *input, term *poly, int n) {
61     char line[MAX_LEN]; // 한 줄 읽어오기
62     for (int i = 0; i < n; i++) {
63         fgets(line, MAX_LEN, input); // 한 줄 읽어오기
64         sscanf(line, "%d\t%d", &poly[i].coef, &poly[i].exp); // 계수와 지수 읽어오기
65     }
66 }
67
68 // 내림차순 정렬(기존 버전)
69 void sort_terms_desc(int acoef[], int aexp[], int n) {
70     int tmp;
71     for (int i = 0; i < n - 1; i++) { // 버블 정렬
72         for (int j = i + 1; j < n; j++) {
73             if (aexp[i] < aexp[j]) { // 지수 비교
74                 tmp = aexp[i]; // 지수 교환
75                 aexp[i] = aexp[j];

```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
69 void sort_terms_desc(int acoef[], int aexp[], int n) {
70     for (int i = 0; i < n - 1; i++) { // 버블 정렬
71         for (int j = i + 1; j < n; j++) {
72             if (aexp[i] < aexp[j]) { // 지수 비교
73                 aexp[j] = tmp;
74                 tmp = acoef[i]; // 계수 교환
75                 acoef[i] = acoef[j];
76                 acoef[j] = tmp;
77             }
78         }
79     }
80 }
81 // 내림차순 정렬(개선된 버전)
82 void sortTermsDesc(polyonomal1 *poly, int start, int finish) {
83     polyonomal1 tmp;
84     for (int i = start; i <= finish - 1; i++) { // 버블 정렬
85         for (int j = i + 1; j <= finish; j++) {
86             if (poly[i].expon < poly[j].expon) { // 지수 비교
87                 tmp = poly[i]; // 지수 교환
88                 poly[i] = poly[j];
89                 poly[j] = tmp;
90             }
91         }
92     }
93 }
94 // 내림차순 정렬(연결리스트)
95 void sortTermsDesc(term s[], int n) {
96     term tmp;
97     for (int i = 0; i < n - 1; i++) { // 버블 정렬
98         for (int j = i + 1; j < n; j++) {
99             if (s[i].exp < s[j].exp) { // 지수 비교
100                 tmp = s[i]; // 지수 교환
101                 s[i] = s[j];
102                 s[j] = tmp;
103             }
104         }
105     }
106 }
107 }
108 }
109 }
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
99 void SortTermsDesc(term s[], int n) {
100     for (int i = 0; i < n - 1; i++) { // 버블 정렬
101     }
102 }
103 // term 배열을 polynomial로 변환(기본 버전)
104 polynomial terms_to_polynomial(int acoef[], int aexp[], int n) {
105     polynomial p; // 구조체 p 선언
106     p.degree = aexp[0]; // 최대 지수 degree
107     // 동적 메모리 할당
108     p.coef = (int *)malloc((p.degree + 1) * sizeof(int));
109     // 초기화
110     for (int i = 0; i <= p.degree; i++) {
111         p.coef[i] = 0; // 계수 초기화
112     }
113     // 계수 입력
114     for (int i = 0; i < n; i++) {
115         int index = p.degree - aexp[i]; // 지수에 따라 인덱스 계산
116         p.coef[index] = acoef[i]; // 계수 저장
117     }
118     return p; // 다항식 반환
119 }
120 // term 배열을 연결리스트 polyPointer로 변환(연결리스트)
121 polyPointer initialPolyNode(int n, term s[]) {
122     // 첫 번째 노드 동적 메모리 할당
123     polyPointer temp = (polyPointer)malloc(sizeof(polyNode));
124     temp->coef = s[0].coef; // 첫 항 계수
125     temp->exp = s[0].exp; // 첫 항 지수
126     temp->link = NULL; // 다음 노드 포인터 초기화
127     // 리스트 시작점 head, 이전 노드 가리키는 prev 포인터
128     polyPointer head = temp, prev = temp;
129     // 나머지 항들 연결리스트 추가
130     for (int i = 1; i < n; i++) {
131         prev = temp; // temp 노드 prev로 설정
132         temp = (polyPointer)malloc(sizeof(polyNode)); // temp 노드 동적 메모리 할당
133         temp->coef = s[i].coef; // 계수 저장
134     }
135 }
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
131 polyPointer initialPolyNode(int n, term s[]){
142     for(int i=1;i<n;++i){
146         temp->exp = s[i].exp; // 지수 저장
147         temp->link = NULL; // 다음 노드 포인터 초기화
148         prev->link = temp; // 이전 노드의 링크를 현재 노드로 설정
149     }
150     return head; // 연결리스트 반환
151 }
152
153 // 다항식 출력(기본버전)
154 void print_polynomial(polynomial p, FILE *output) {
155     int count = 0; // 항 개수 카운트
156     for (int i = 0; i <= p.degree; i++) {
157         if (p.coef[i] != 0) { // 계수가 0이 아니면
158             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
159             fprintf(output, "%dx^%d", p.coef[i], p.degree - i); // 계수와 지수 출력
160             count++; // 항 개수 증가
161         }
162     }
163     fprintf(output, "\n"); // 줄 바꿈
164 }
165
166 // 다항식 출력(개선된 버전)
167 void printPolynomial(polynomial1 *poly, int start, int finish, FILE *output) {
168     int count = 0; // 항 개수 카운트
169     for (int i = start; i <= finish; i++) {
170         if (poly[i].coef != 0) { // 계수가 0이 아니면
171             if (count > 0) fprintf(output, " + "); // 첫 항이 아니면 +
172             fprintf(output, "%dx^%d", poly[i].coef, poly[i].expon); // 계수와 지수 출력
173             count++; // 항 개수 증가
174         }
175     }
176     fprintf(output, "\n"); // 줄 바꿈
177 }
178
179 //다항식 출력(연결리스트)
180 int printPolyNode(polyPointer head, FILE *output){
181     polyPointer temp = head; // 연결리스트 포인터
182     while(temp != NULL){ // 연결리스트 순회
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
179 int printPolyNode(polyPointer head, FILE *output){
181     while(temp != NULL){ // 연결리스트 순회
182         if(temp->link != NULL){ // 마지막 항이 아니면
183             fprintf(output, "%dx^%d + ", temp->coef, temp->exp); // 계수와 지수 출력
184         }else{ // 마지막 항이면
185             fprintf(output, "%dx^%d", temp->coef, temp->exp); // 계수와 지수 출력
186         }
187         temp = temp->link; // 다음 노드로 이동
188     }
189     fprintf(output, "\n"); // 줄 바꿈
190     return 0;
191 }
192
193 // Zero: 다항식을 0으로 초기화
194 polynomial Zero(int degree){
195     polynomial p; // 다항식 구조체
196     p.degree = degree; // 최대 차수
197     p.coef = (int *)malloc((p.degree + 1)*sizeof(int)); // 동적 메모리 할당
198     for(int i = 0; i <= p.degree; i++){
199         p.coef[i] = 0; // 계수 초기화
200     }
201     return p; // 다항식 반환
202 }
203
204 // IsZero: 모든 계수가 0이면 true
205 bool IsZero(polynomial p){
206     for(int i = 0; i <= p.degree; i++){ // 계수 체크
207         if(p.coef[i] != 0){ // 계수가 0이 아니면
208             return false; // 비어있지 않으면 false
209         }
210     }
211     return true; // 비어있으면 true
212 }
213
214 // Coef: 특정 차수의 계수를 반환
215 int Coef(polynomial p, int expon){
216     int index = p.degree - expon; // 지수에 해당하는 인덱스
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
216 int Coef(polyomial p, int expon){
217     if(index < 0 || index > p.degree){ // 유효한 인덱스 범위 체크
218         return 0; // 해당 차수가 없으면 0 반환
219     }
220     return p.coef[index]; // 계수 반환
221 }
222
223 // Lead_Exp: 최고 차항의 지수를 반환
224 int Lead_Exp(polyomial p){
225     for(int i = 0; i <= p.degree; i++){ // 최고 차항 찾기
226         if(p.coef[i] != 0){ // 계수가 0이 아니면
227             return p.degree - i; // 최고 차항의 지수 반환
228         }
229     }
230     return -1; // 다항식이 0이면 -1 반환
231 }
232
233 // Attach: 다항식에 항을 추가(기존 버전)
234 polynomial Attach(polyomial p, int coef, int expon){
235     int index = p.degree - expon; // 지수에 해당하는 인덱스
236     if(p.coef[index] != 0){ // 계수가 0이 아니면
237         printf("ERROR"); // 이미 존재하는 항이면 오류
238     }
239     else{ // 계수가 0이면
240         p.coef[index] = coef; // 계수 추가
241     }
242     return p; // 다항식 반환
243 }
244
245 // attach: 새로운 항 첨가(개선된 버전)
246 void attach(int coef, int expon, int* avail, polynomial1 **poly) {
247     *poly = realloc(*poly, sizeof(polynomial1) * (*avail + 1)); // 동적 메모리 재할당
248     (*poly)[*avail].coef = coef; // 계수 추가
249     (*poly)[*avail].expon = expon; // 지수 추가
250     (*avail)++; // 인덱스 증가
251 }
252
253 // attachPolyNode: 새로운 항 첨가(연결리스트)
254 int attachPolyNode(int coef, int exp, polyPointer* rear){
255     polyPointer temp = (polyPointer)malloc(sizeof(polyNode)); // 새로운 노드 생성
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
253 int attachPolyNode(int coef, int exp, polyPointer* rear){
254     temp->coef = coef; // 계수 추가
255     temp->exp = exp; // 지수 추가
256     temp->link = NULL; // 다음 노드 포인터 초기화
257
258     if (*rear) { // 연결리스트가 비어있지 않으면
259         (*rear)->link = temp; // 이전 노드의 링크를 새로운 노드로 설정
260     }
261     *rear = temp; // 현재 노드를 rear로 설정
262     return 0;
263 }
264
265 // Remove: 다항식에 항 제거
266 polynomial Remove(polyomial p, int expon){
267     int index = p.degree - expon; // 지수에 해당하는 인덱스
268     if(p.coef[index] != 0){ // 계수가 0이 아니면
269         p.coef[index] = 0; // 계수 제거
270     }
271     else{ // 계수가 0이면
272         printf("ERROR"); // 이미 존재하지 않는 항이면 오류
273     }
274     return p; // 다항식 반환
275 }
276
277 // COMPARE: 두 항의 지수 비교
278 int COMPARE(int exp1, int exp2){
279     if(exp1 < exp2) // a1 < a2
280         return -1;
281     else if(exp1 == exp2) // a1 = a2
282         return 0;
283     else // a1 > a2
284         return 1;
285 }
286
287 // addPolyNode: 두 다항식 덧셈(연결리스트)
288 polyPointer addPolyNode(polyPointer a, polyPointer b){
289     // 다항식 결과의 시작과 마지막 가리킬 헤드와 리어 포인터
290     polyPointer head = NULL, rear = NULL;
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
289 polyPointer addPolyNode(polyPointer a, polyPointer b){
292     int sum;
293     rear = (polyPointer)malloc(sizeof(polyNode)); // rear 노드 동적 메모리 할당
294     rear->coef = 0; // 계수 초기화
295     rear->exp = 0; // 지수 초기화
296     rear->link = NULL; // 다음 노드 포인터 초기화
297     head = rear; // head 포인터가 rear를 가리킴
298     while(a != NULL && b != NULL){ // 두 다항식이 비어있지 않을 때까지 반복
299         switch(COMPARE(a->exp, b->exp)){ // a와 b의 지수 비교
300             case -1: // a < b
301                 attachPolyNode(b->coef, b->exp, &rear); // b의 항을 추가
302                 b = b->link; // b의 항을 rear 다음에 추가
303                 break;
304             case 0: // a = b
305                 sum = a->coef + b->coef; // 동일 차수 계수 합
306                 if(sum){ // 합이 0이 아니면
307                     attachPolyNode(sum, a->exp, &rear); //결과 다항식에 합 추가
308                 }
309                 a = a->link; // a의 항을 rear 다음에 추가
310                 b = b->link; // b의 항을 rear 다음에 추가
311                 break;
312             case 1: // a > b
313                 attachPolyNode(a->coef, a->exp, &rear); // a의 항을 추가
314                 a = a->link; // a의 항을 rear 다음에 추가
315                 break;
316         }
317     }
318     for(;a;a=a->link){ //a에 남은 항을 결과에 추가
319         attachPolyNode(a->coef, a->exp, &rear); // a의 항을 rear 다음에 추가
320     }
321     for(;b;b=b->link){ // b에 남은 항을 결과에 추가
322         attachPolyNode(b->coef, b->exp, &rear); // b의 항을 rear 다음에 추가
323     }
324     rear->link = NULL; // 마지막 노드의 링크를 NULL로 설정
325     return head->link; // 결과 다항식 반환
327 }
328
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
329 //freePolyNode: 다항식 메모리 해제(연결리스트)
330 int freePolyNode(polyPointer head){
331     polyPointer temp = head,next=NULL; // head 가리키는 temp, 다음 노드 가리킬 next 포인터
332     // 연결리스트 순회하며 메모리 해제
333     while(temp != NULL){ //현재 노드가 NULL이 아닐 때까지 반복
334         next = temp->link; // 다음 노드 저장
335         free(temp); // 현재 노드 메모리 해제
336         temp = next; // 다음 노드로 이동
337     }
338     return 0;
339 }
340
341 int main(int argc, char *argv[]){
342     //시작 시간 측정(파일 열기)
343     double start_time = current_millis();
344
345     //input 파일 읽어오기
346     FILE *input = fopen(argv[1], "r");
347     char line[MAX_LEN];
348     //output 파일 열기
349     FILE *output = fopen(argv[2], "w");
350
351     // 종료 시간 측정(파일 열기)
352     double end_time = current_millis();
353
354     //-----기존 버전 시작
355
356     //시작 시간 측정(기존 버전)
357     double start_time0 = current_millis();
358
359     //첫 줄 읽기: 지수 계수 2개
360     fgets(line, MAX_LEN, input);
361     int n1, n2;
362     sscanf(line, "%d\t%d", &n1, &n2);
363
364     //배열 alcoef, alexp 선언
365     int* alcoef = (int*)malloc(sizeof(int) * n1);
366
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
367     int* a1exp = (int*)malloc(sizeof(int) * n1);
368     //배열 a2coef, a2exp 선언
369     int* a2coef = (int*)malloc(sizeof(int) * n2);
370     int* a2exp = (int*)malloc(sizeof(int) * n2);
371
372     // n1 줄들 읽어와 s1 저장
373     read_terms(input, a1coef, a1exp, n1);
374     // n2 줄들 읽어와 s2 저장
375     read_terms(input, a2coef, a2exp, n2);
376
377     //배열 s1 내림차순 정렬
378     sort_terms_desc(a1coef, a1exp, n1);
379     //배열 s2 내림차순 정렬
380     sort_terms_desc(a2coef, a2exp, n2);
381
382     //구조체 a1 초기화 후 s1 저장
383     polynomial a1 = terms_to_polynomial(a1coef, a1exp, n1);
384     //구조체 a2 초기화 후 s2 저장
385     polynomial a2 = terms_to_polynomial(a2coef, a2exp, n2);
386
387     //다항식 a1 출력
388     print_polynomial(a1, output);
389     //다항식 a2 출력
390     print_polynomial(a2, output);
391
392
393     //구조체 d 선언
394     polynomial d;
395     // 다항식 덧셈(초기버전)
396     d = Zero(a1.degree > a2.degree ? a1.degree : a2.degree);
397     int sum = 0;
398     while(!IsZero(a1) && !IsZero(a2)){ // 두 다항식이 비어있지 않을 때까지 반복
399         switch (COMPARE(Lead_Exp(a1), Lead_Exp(a2))){ // a1과 a2의 최고차항 지수 비교
400             case -1: // a1 < a2
401                 d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
402                 a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
403                 break;
404             case 0: // a1 = a2
405                 sum = Coef(a1, Lead_Exp(a1)) + Coef(a2, Lead_Exp(a2)); // 동일 차수 계수 합
406                 if(sum != 0){ // 합이 0이 아니면
407                     d = Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
408                 }
409                 else{ // 합이 0이면
410                     d = Attach(d, 0, Lead_Exp(a1)); // d에 0 추가
411                 }
412                 if(sum){ // 합이 0이 아니면
413                     Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
414                     a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
415                     a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
416                 }
417                 break;
418             case 1: // a1 > a2
419                 d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
420                 a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
421                 break;
422         }
423     }
424     //a1 또는 a2 나머지 항을 d에 삽입
425     while(!IsZero(a1)){ // a1이 비어있지 않으면
426         d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
427         a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
428     }
429     while(!IsZero(a2)){ // a2가 비어있지 않으면
430         d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
431         a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
432     }
433
434     // 다항식 d 출력
435     print_polynomial(d, output);
436
437     // 종료 시간 측정(기준 버전)
438     double end_time0 = current_millis();
439 }
```

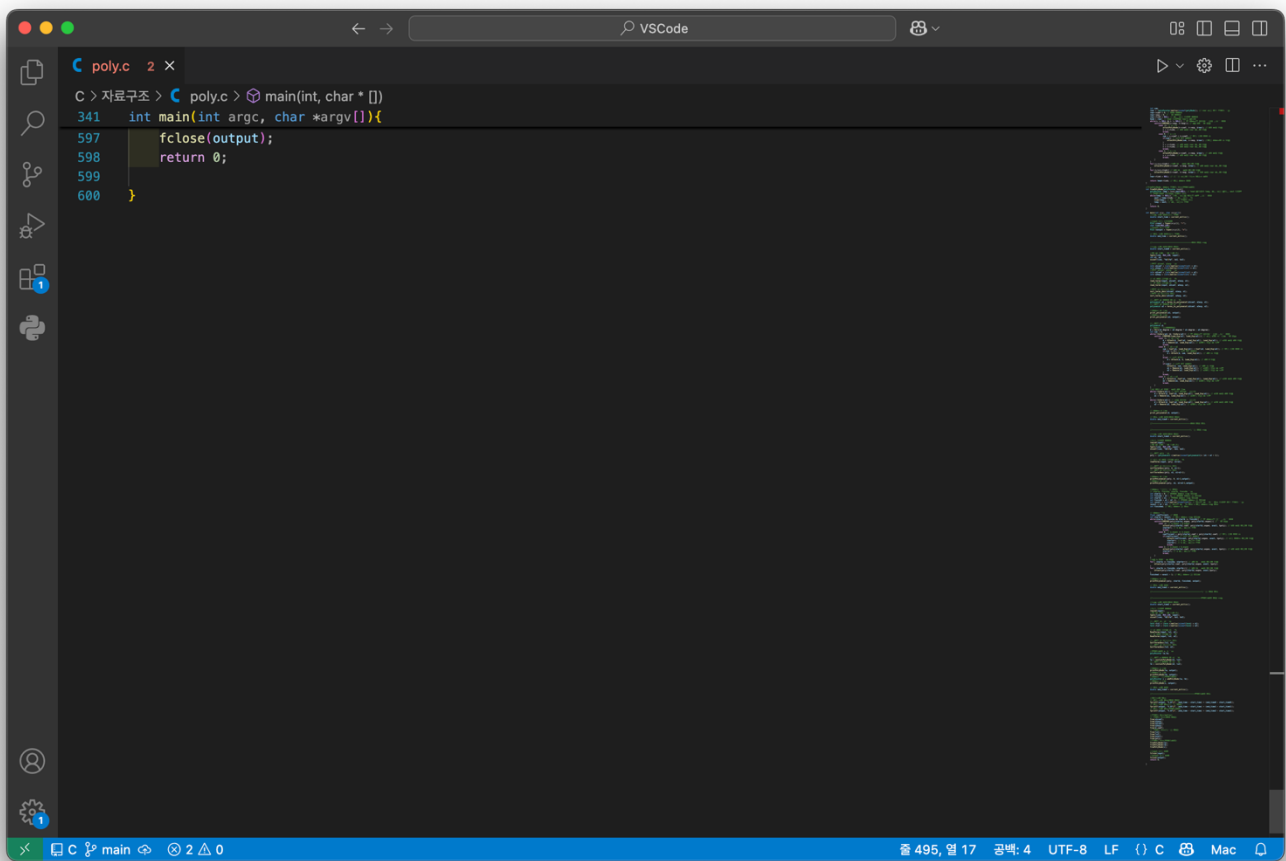
```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
367     int* a1exp = (int*)malloc(sizeof(int) * n1);
368     //배열 a2coef, a2exp 선언
369     int* a2coef = (int*)malloc(sizeof(int) * n2);
370     int* a2exp = (int*)malloc(sizeof(int) * n2);
371
372     // n1 줄들 읽어와 s1 저장
373     read_terms(input, a1coef, a1exp, n1);
374     // n2 줄들 읽어와 s2 저장
375     read_terms(input, a2coef, a2exp, n2);
376
377     //배열 s1 내림차순 정렬
378     sort_terms_desc(a1coef, a1exp, n1);
379     //배열 s2 내림차순 정렬
380     sort_terms_desc(a2coef, a2exp, n2);
381
382     //구조체 a1 초기화 후 s1 저장
383     polynomial a1 = terms_to_polynomial(a1coef, a1exp, n1);
384     //구조체 a2 초기화 후 s2 저장
385     polynomial a2 = terms_to_polynomial(a2coef, a2exp, n2);
386
387     //다항식 a1 출력
388     print_polynomial(a1, output);
389     //다항식 a2 출력
390     print_polynomial(a2, output);
391
392
393     //구조체 d 선언
394     polynomial d;
395     // 다항식 덧셈(초기버전)
396     d = Zero(a1.degree > a2.degree ? a1.degree : a2.degree);
397     int sum = 0;
398     while(!IsZero(a1) && !IsZero(a2)){ // 두 다항식이 비어있지 않을 때까지 반복
399         switch (COMPARE(Lead_Exp(a1), Lead_Exp(a2))){ // a1과 a2의 최고차항 지수 비교
400             case -1: // a1 < a2
401                 d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
402                 a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
403                 break;
404             case 0: // a1 = a2
405                 sum = Coef(a1, Lead_Exp(a1)) + Coef(a2, Lead_Exp(a2)); // 동일 차수 계수 합
406                 if(sum != 0){ // 합이 0이 아니면
407                     d = Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
408                 }
409                 else{ // 합이 0이면
410                     d = Attach(d, 0, Lead_Exp(a1)); // d에 0 추가
411                 }
412                 if(sum){ // 합이 0이 아니면
413                     Attach(d, sum, Lead_Exp(a1)); // d에 합 추가
414                     a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
415                     a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
416                 }
417                 break;
418             case 1: // a1 > a2
419                 d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
420                 a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
421                 break;
422         }
423     }
424     //a1 또는 a2 나머지 항을 d에 삽입
425     while(!IsZero(a1)){ // a1이 비어있지 않으면
426         d = Attach(d, Coef(a1, Lead_Exp(a1)), Lead_Exp(a1)); // a1의 항을 d에 추가
427         a1 = Remove(a1, Lead_Exp(a1)); // a1에서 해당 항 제거
428     }
429     while(!IsZero(a2)){ // a2가 비어있지 않으면
430         d = Attach(d, Coef(a2, Lead_Exp(a2)), Lead_Exp(a2)); // a2의 항을 d에 추가
431         a2 = Remove(a2, Lead_Exp(a2)); // a2에서 해당 항 제거
432     }
433
434     // 다항식 d 출력
435     print_polynomial(d, output);
436
437     // 종료 시간 측정(기준 버전)
438     double end_time0 = current_millis();
439 }
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
439
440 //-----기존 버전 종료
441
442 //-----개선된 버전 시작
443
444 //시작 시간 측정(기존 버전)
445 double start_time1 = current_millis();
446
447 //파일 포인터 초기화
448 rewind(input);
449 //첫 줄 읽기: 지수 개수 2개
450 fgets(line, MAX_LEN, input);
451 sscanf(line, "%d\t%d", &n1, &n2);
452
453 //구조체 poly 선언
454 poly = (polynomial1 *)malloc(sizeof(polynomial1)* (n1 + n2 + 1));
455
456 // n1과 n2 줄들 읽어와 poly 저장
457 readTerms(input, poly, n1+n2);
458
459 //구조체 s1 내림차순 정렬
460 sortTermsDesc(poly, 0, n1-1);
461 //구조체 s2 내림차순 정렬
462 sortTermsDesc(poly, n1, n1+n2-1);
463
464 //다항식 a1 출력
465 printPolynomial(poly, 0, n1-1,output);
466 //다항식 a2 출력
467 printPolynomial(poly, n1, n1+n2-1,output);
468
469
470 //다항식 덧셈(개선된 버전)
471 // starta, finisha, startb, finishb 선언
472 int starta = 0; // 첫번째 다항식 시작 인덱스
473 int finisha = n1 - 1; // 첫번째 다항식 끝 인덱스
474 int startb = n1; // 두번째 다항식 시작 인덱스
475
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
476 int finishb = n1 + n2 - 1; // 두번째 다항식 끝 인덱스
477 int *avail = (int*)malloc(sizeof(int)); // 새로운 항 저장할 위치 포인터 동적 메모리 할당
478 *avail = n1 + n2; // 새로운 항 저장 위치 = 결과 다항식 시작 위치
479 int finished; // 결과 다항식 끝 위치
480
481 // 다항식 덧셈
482 float coefficient; // 계수
483 int startd = *avail; // 결과 다항식 시작 인덱스
484 while(starta <= finisha && startb <= finishb){ // 두 다항식이 끝날 때까지 반복
485     switch(COMPARE(poly[starta].expon, poly[startb].expon)){ // 지수 비교
486         case -1: // a.expon < b.expon
487             attach(poly[startb].coef, poly[startb].expon, avail, &poly); // b의 항을 결과에 추가
488             startb++; // b 다음 항으로 이동
489             break;
490         case 0: // a.expon == b.expon
491             coefficient = poly[starta].coef + poly[startb].coef; // 동일 차수 계수 합
492             if(coefficient) // 합이 0이 아니면
493                 attach(coefficient, poly[starta].expon, avail, &poly); // 합한 계수를 결과에 추가
494             starta++; // a 다음 항으로 이동
495             startb++; // b 다음 항으로 이동
496             break;
497         case 1: // a.expon > b.expon
498             attach(poly[starta].coef, poly[starta].expon, avail, &poly); // a의 항을 결과에 추가
499             starta++; // a 다음 항으로 이동
500             break;
501     }
502 }
503 //a와 b 나머지 항 첨가
504 for(; starta <= finisha; starta++){ // a에 남은 항을 결과에 추가
505     attach(poly[starta].coef, poly[starta].expon, avail, &poly);
506 }
507 for(; startb <= finishb; startb++){ // b에 남은 항을 결과에 추가
508     attach(poly[startb].coef, poly[startb].expon, avail, &poly);
509 }
510 finished = *avail - 1; // 결과 다항식 끝 인덱스
511
512
```

```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
513     //다항식 d 출력
514     printPolynomial(poly, startd, finished, output);
515
516     // 종료 시간 측정
517     double end_time1 = current_millis();
518
519     //-----개선된 버전 종료
520
521     //-----연결리스트 버전 시작
522
523     //시작 시간 측정(기본 버전)
524     double start_time2 = current_millis();
525
526     //파일 포인터 초기화
527     rewind(input);
528     //첫 줄 읽기: 지수 개수 2개
529     fgets(line, MAX_LEN, input);
530     sscanf(line, "%d\t%d", &n1, &n2);
531
532     //구조체 s1, s2 선언
533     term *ls1 = (term *)malloc(sizeof(term) * n1);
534     term *ls2 = (term *)malloc(sizeof(term) * n2);
535
536     // n1 줄들 읽어와 s1 저장
537     ReadTerms(input, ls1, n1);
538     // n2 줄들 읽어와 s2 저장
539     ReadTerms(input, ls2, n2);
540
541     //구조체 s1 내림차순 정렬
542     SortTermsDesc(ls1, n1);
543     //구조체 s2 내림차순 정렬
544     SortTermsDesc(ls2, n2);
545
546     //연결리스트 a, b 선언
547     polyPointer la, lb;
548
549 }
```

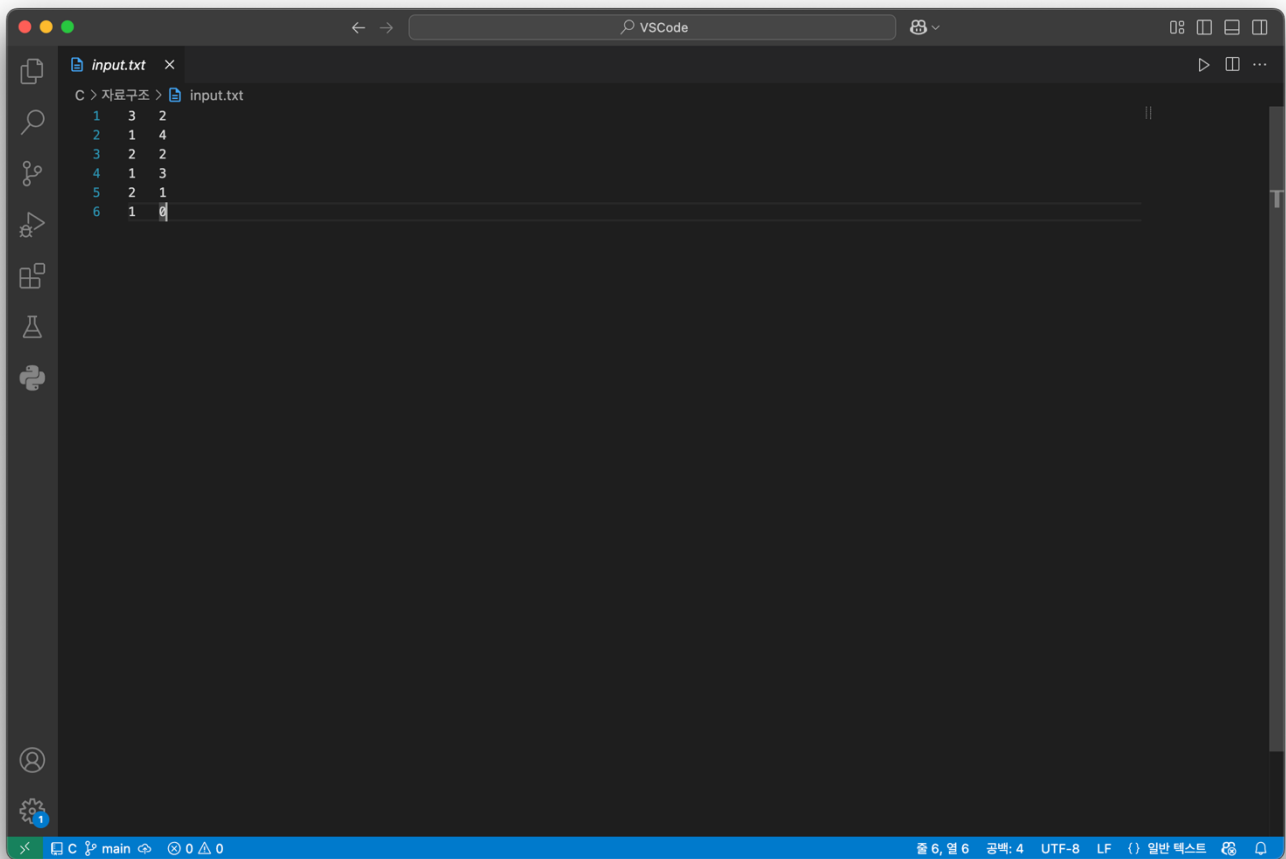
```
poly.c 2 X
C > 자료구조 > C poly.c > main(int, char* [])
341 int main(int argc, char *argv[]){
560     polyPointer c = addPolyNode(la, lb);
561     //다항식 c 출력
562     printPolyNode(c, output);
563
564     // 종료 시간 측정
565     double end_time2 = current_millis();
566
567     //-----연결리스트 종료
568
569     //수행시간 기록
570     // 수행 시간 기록(기본 버전)
571     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time0 - start_time0));
572     // 수행 시간 기록(개선된 버전)
573     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time1 - start_time1));
574     // 수행 시간 기록(연결리스트)
575     fprintf(output, "%.6f\t", (end_time - start_time) + (end_time2 - start_time2));
576
577     //메모리 해제(malloc)
578     // 메모리 해제(기본 버전)
579     free(alcoef);
580     free(alcoef);
581     free(a2coef);
582     free(a2coef);
583     free(d.coef);
584     // 메모리 해제(개선된 버전)
585     free(ls1);
586     free(ls2);
587     free(avail);
588     free(poly);
589     //메모리 해제(연결리스트)
590     freePolyNode(la);
591     freePolyNode(lb);
592     freePolyNode(c);
593
594     //input 파일 닫기
595     fclose(input);
596     //output 파일 닫기
597 }
```



02. 코드 실행 방법

- 1) macOS version

i) input.txt 파일

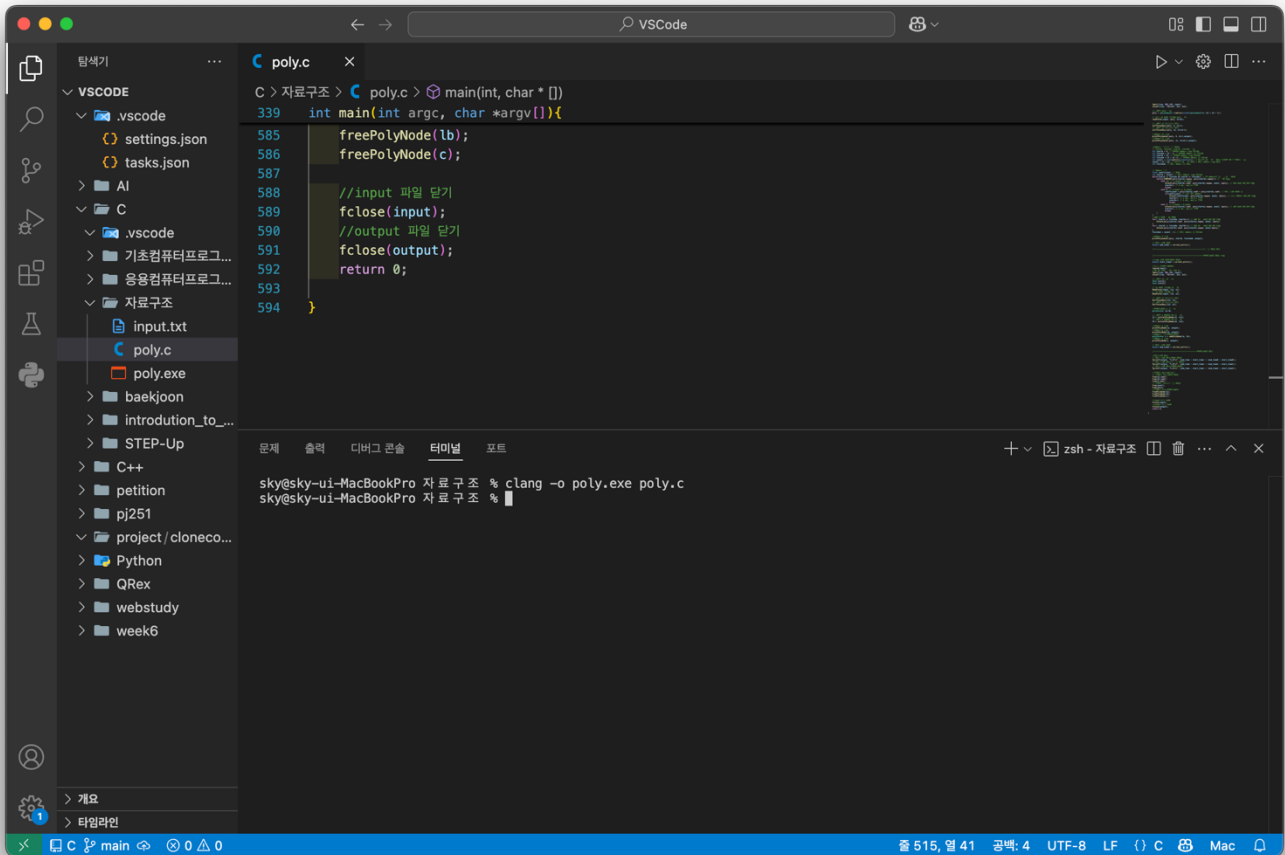


The image shows a screenshot of the Visual Studio Code (VS Code) editor running on a macOS system. The window title is "VSCode". The file explorer on the left shows a file named "input.txt" in the "C > 자료구조" directory. The editor displays the contents of "input.txt", which consists of 6 lines of numbers:

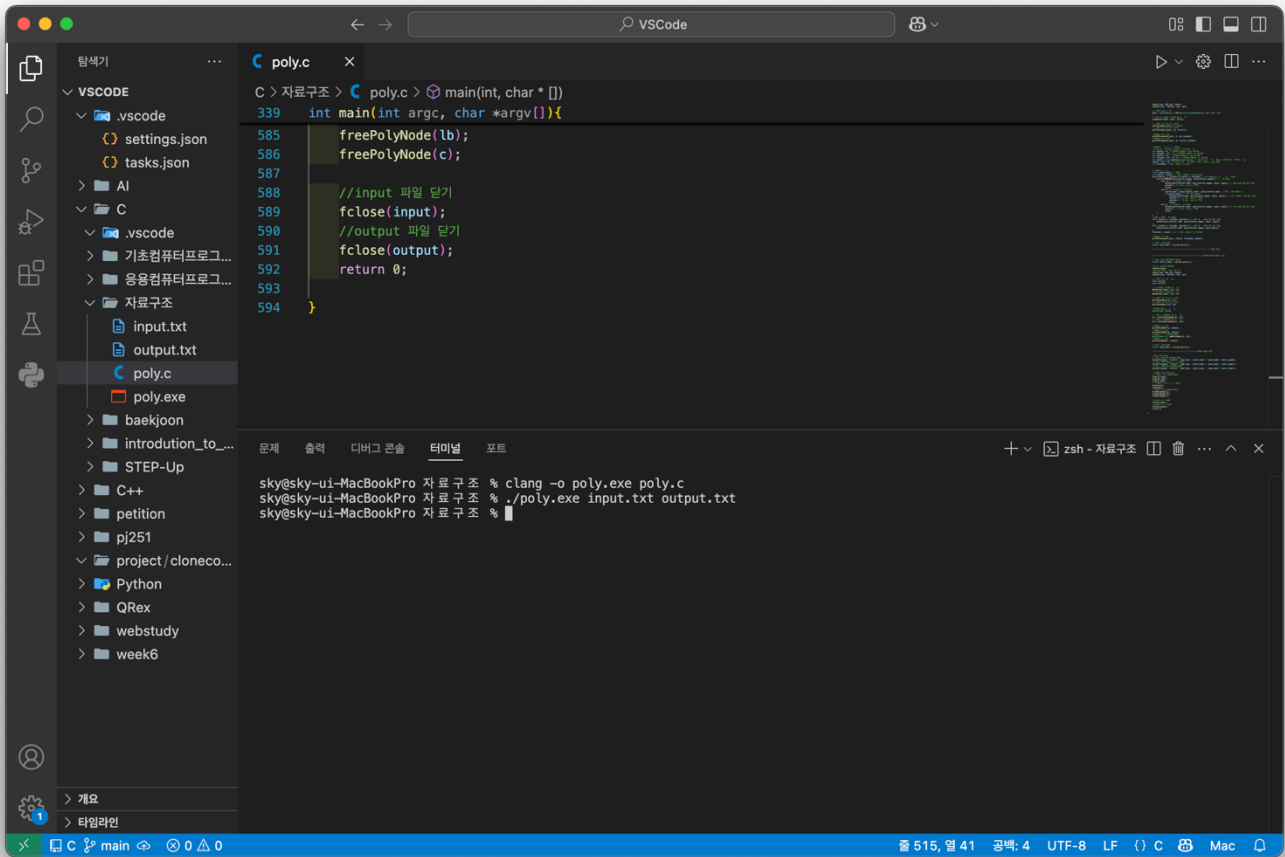
```
1 3 2
2 1 4
3 2 2
4 1 3
5 2 1
6 1
```

The status bar at the bottom indicates the file is "main", has 0 errors and 0 warnings, and is using the "UTF-8" encoding and "LF" line endings.

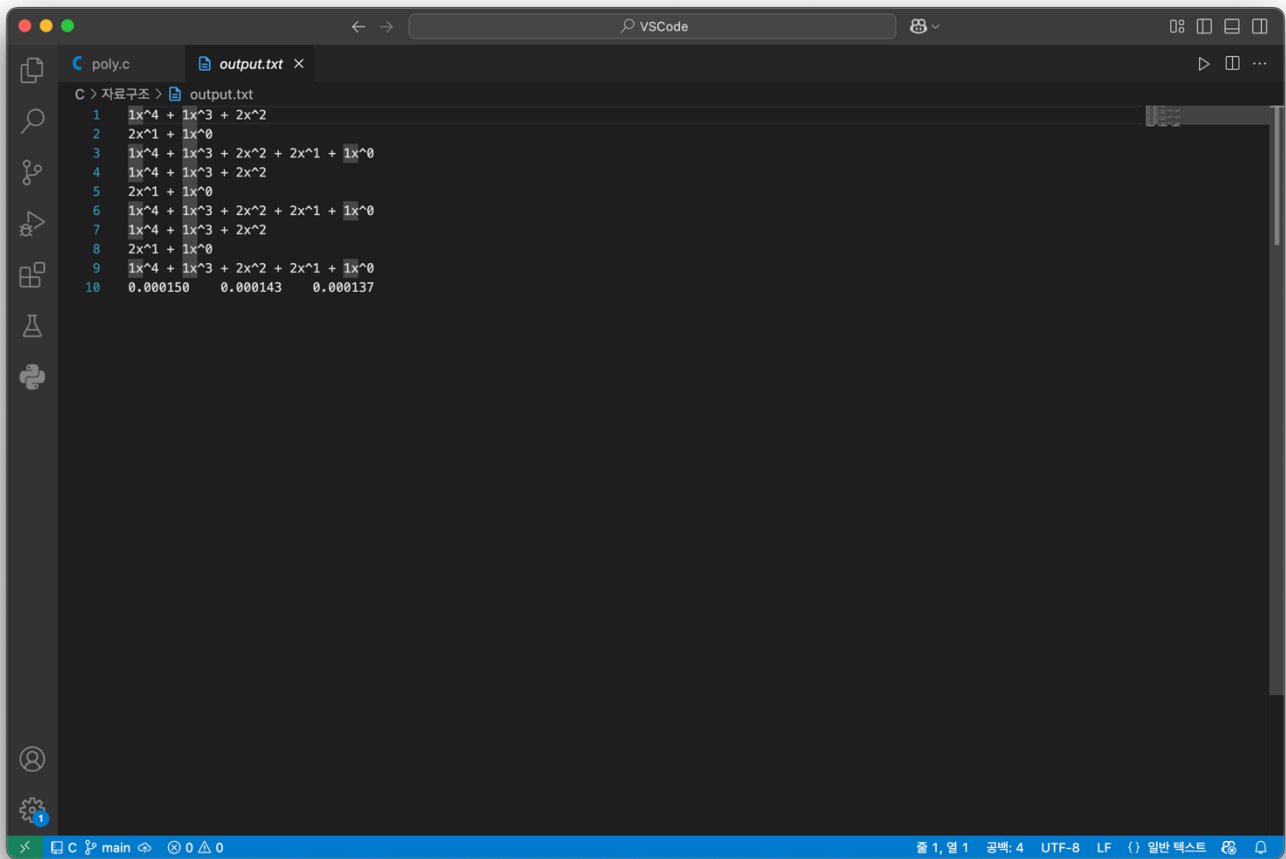
ii) 컴파일: `clang -o poly.exe poly.c`



iii) 실행: `./poly.exe input.txt output.txt`



iv) output.txt 파일 생성



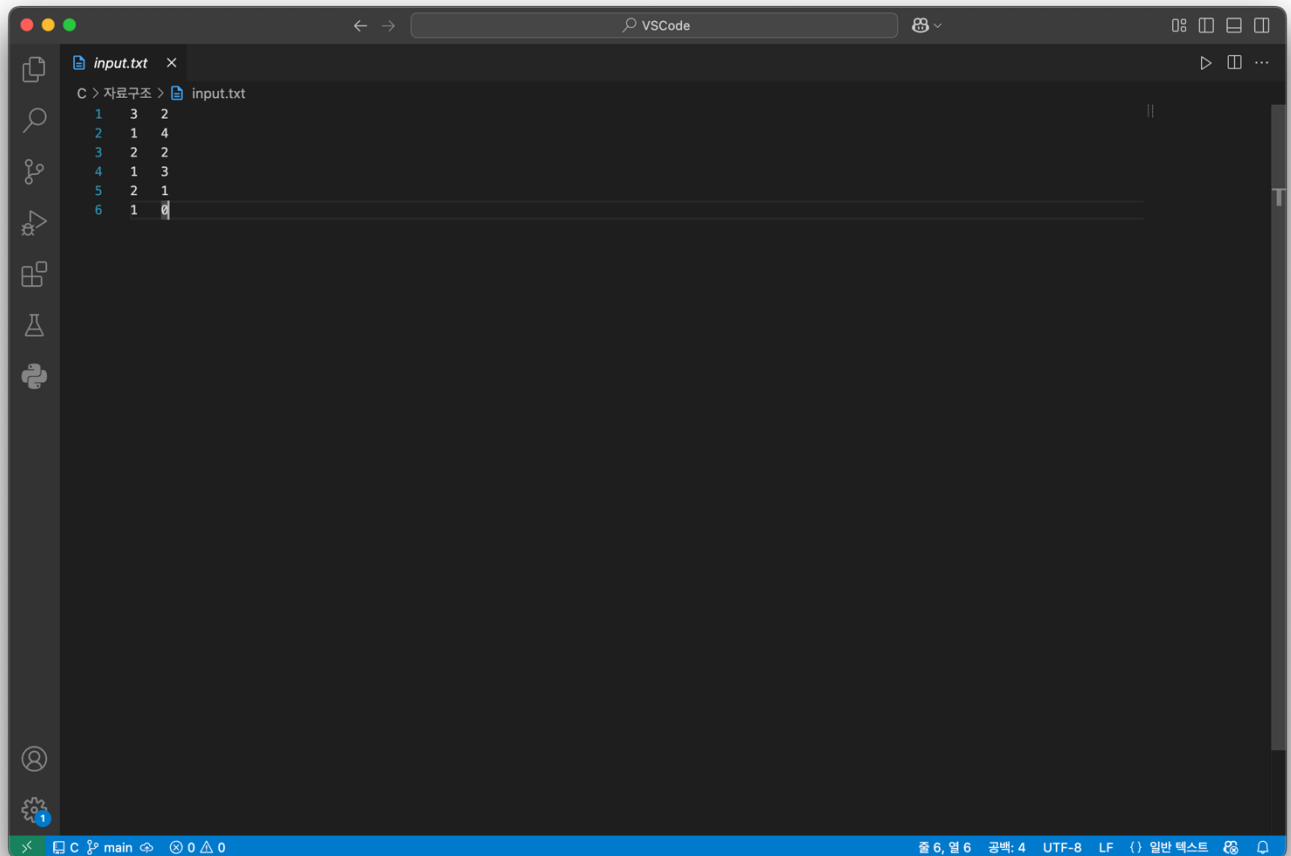
The screenshot shows the Visual Studio Code (VS Code) editor interface. The file explorer on the left shows a project named 'poly.c' with a file named 'output.txt' selected. The editor window displays the contents of 'output.txt' with the following text:

```
1 1x^4 + 1x^3 + 2x^2
2 2x^1 + 1x^0
3 1x^4 + 1x^3 + 2x^2 + 2x^1 + 1x^0
4 1x^4 + 1x^3 + 2x^2
5 2x^1 + 1x^0
6 1x^4 + 1x^3 + 2x^2 + 2x^1 + 1x^0
7 1x^4 + 1x^3 + 2x^2
8 2x^1 + 1x^0
9 1x^4 + 1x^3 + 2x^2 + 2x^1 + 1x^0
10 0.000150 0.000143 0.000137
```

The status bar at the bottom indicates the current file is 'main' in the 'C' language, with 0 errors and 0 warnings. The encoding is UTF-8, and the line ending is LF.

- 2) window version

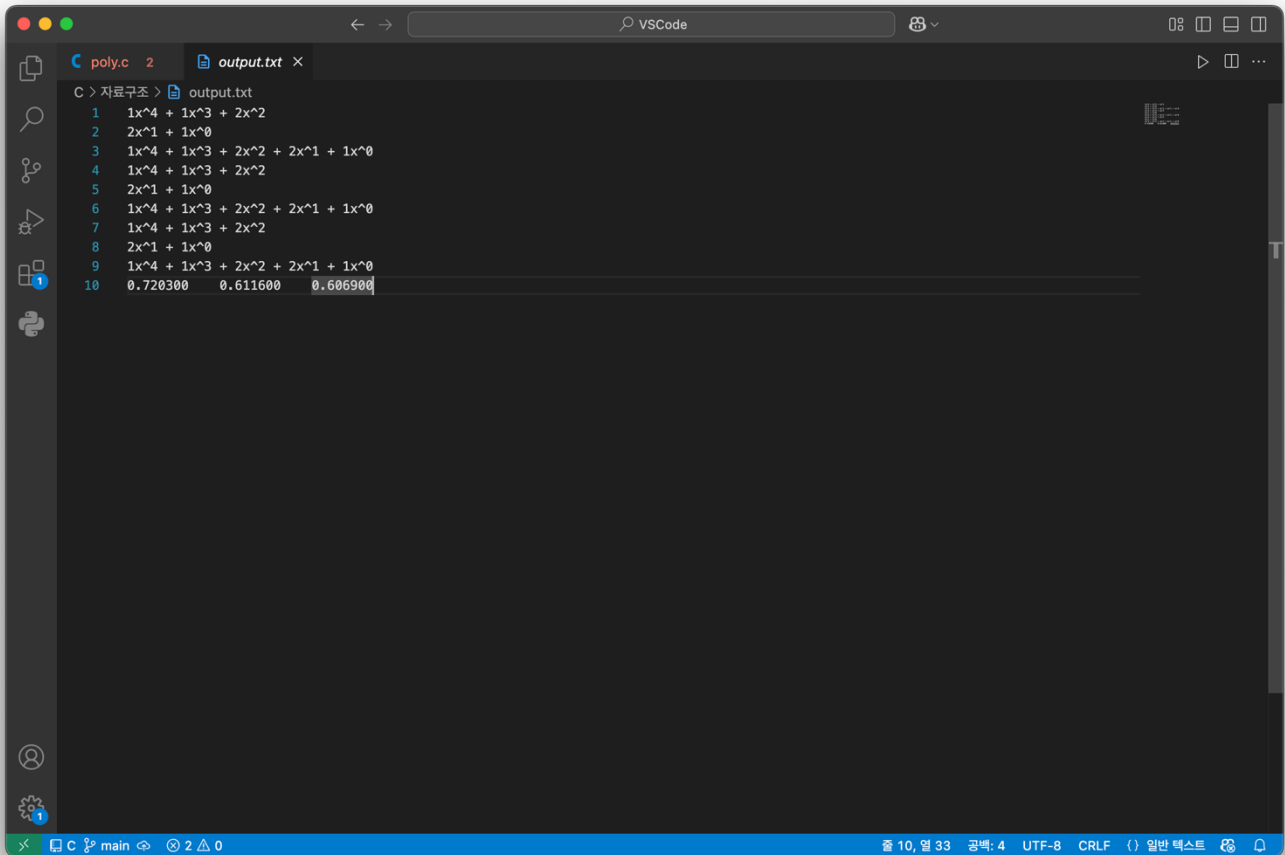
i) input.txt 파일 실행



ii) 컴파일: `clang poly.c -o poly.exe`

iii) 실행: `./poly.exe input.txt output.txt`

iv) output.txt 파일 생성



03. TA의 컴퓨터에서 실행하기 위한 방법

- 컴파일러 버전: `gcc --version`

```
PS C:\Users\오다영\OneDrive\바탕 화면\여민> gcc --version
gcc.exe (MinGW.org GCC-6.3.0-1) 6.3.0
Copyright (C) 2016 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

04. 유의사항

본인이 macOS를 사용 중으로 이를 이용하여 생성한 실행파일(poly.exe)이 window에서 동작하지 않아 타 학우의 window 노트북으로 생성한 실행파일을 제출합니다.

위에 첨부한 컴파일러 버전과 제출한 c파일(poly.c)는 모두 window에 맞춰져 작성된 상태입니다.